

Advanced 3D Game Development CA2 Report

Name: Wolfgang Romanowski

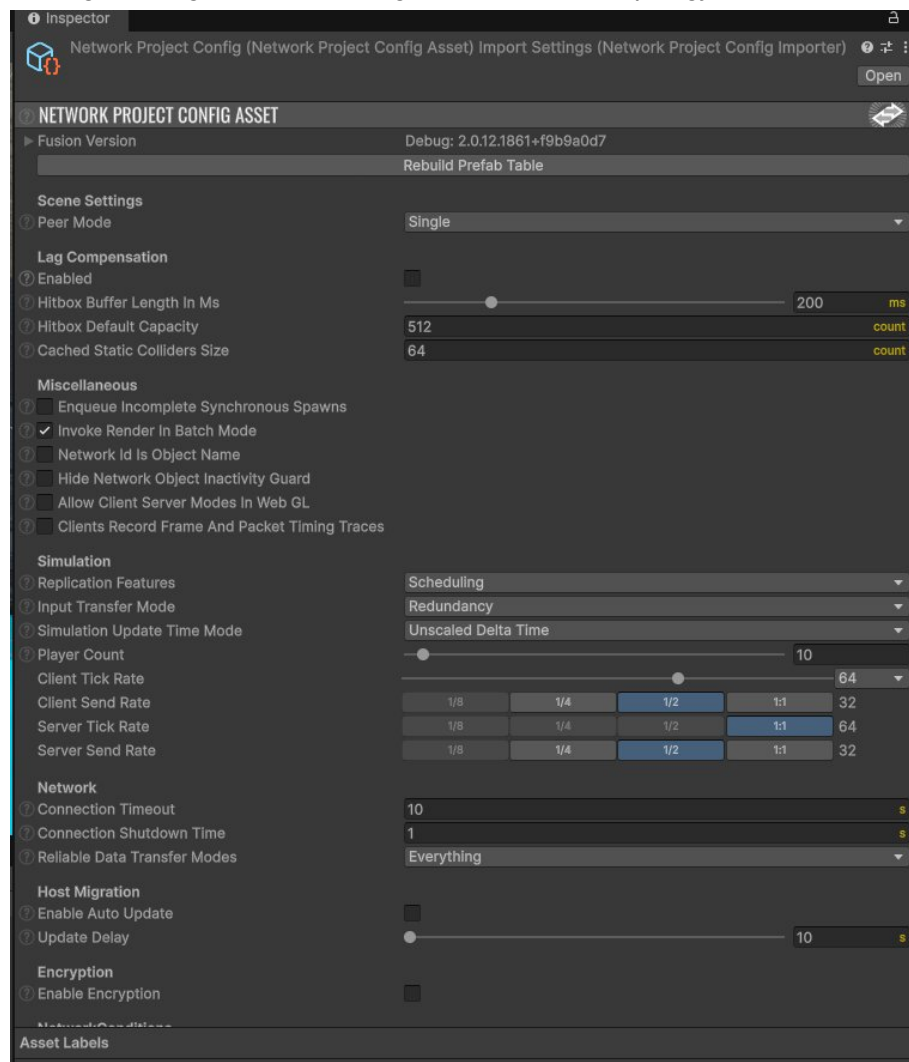
Student Number: 20101931

Video: https://youtu.be/FN_ch8vtokA

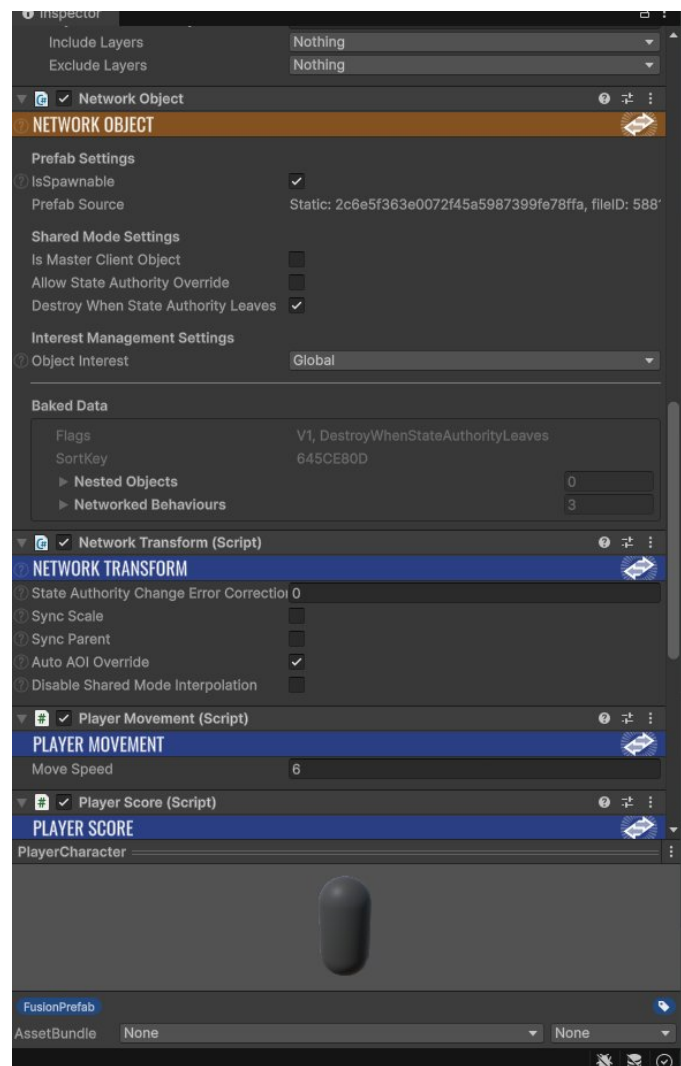
Repository: <https://github.com/Wolfgang-Romanowski/GameDev-Unity-CA-Sem2/tree/feature/ca2-network>

A1 Session Setup and Spawn Baseline

NetworkProjectConfig showing Peer Mode: Single for Host/Client topology. Fusion version 2.0.12.

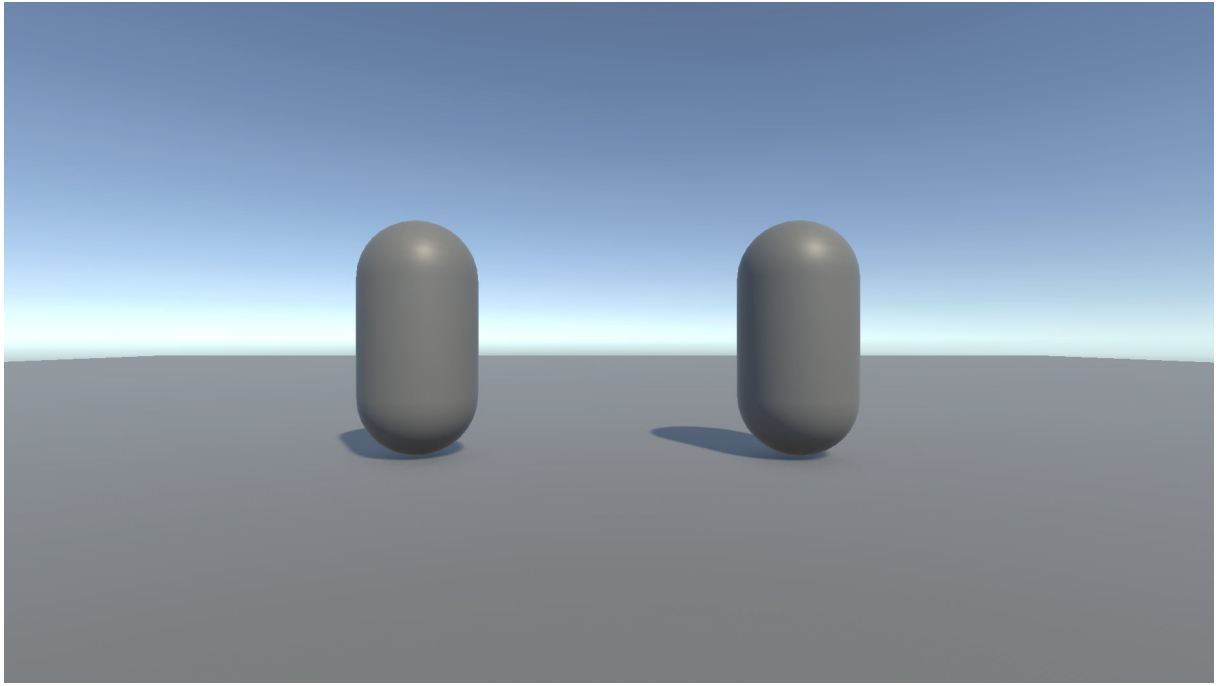


PlayerCharacter prefab with NetworkObject, NetworkTransform, PlayerMovement, and PlayerScore components. Registered in the NetworkProjectConfig prefab table via Rebuild Prefab Table.



The Spawner script implements `INetworkRunnerCallbacks`. On session join, `OnPlayerJoined` calls `Runner.Spawn(playerPrefab, spawnPos, Quaternion.identity, player)` where the fourth parameter assigns `InputAuthority` to the joining client. `StateAuthority` is held by the host over all spawned and scene-placed `NetworkObjects`.

Two-client baseline: standalone build as host, Unity editor as client. Both capsules visible and moving independently.



A2 Networked Pickup Feature (Option B)

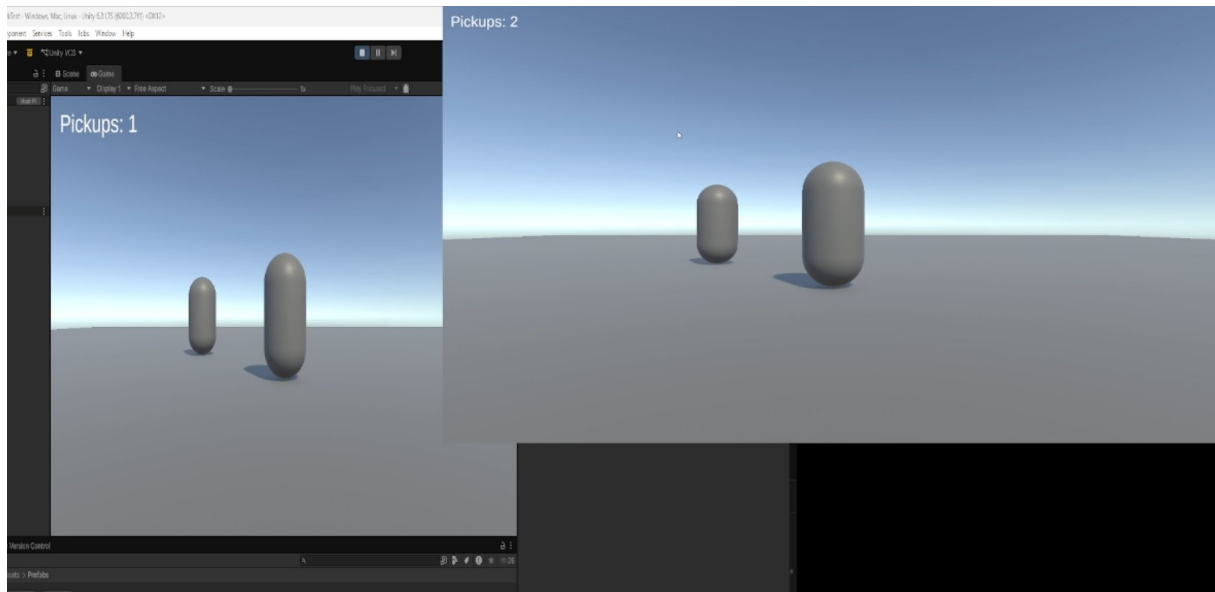
Pickup orbs are scene-placed NetworkObjects with trigger colliders. When a player enters the trigger, the host sets `IsPickedUp` to true and increments the player's Score. Both are [Networked] properties replicated automatically to all clients.

`HasStateAuthority` guard in `PickupItem.OnTriggerEnter` prevents both clients from processing the same collision. Only the host writes to [Networked] properties. `IsPickedUp` check prevents re-collection of already-picked-up orbs.

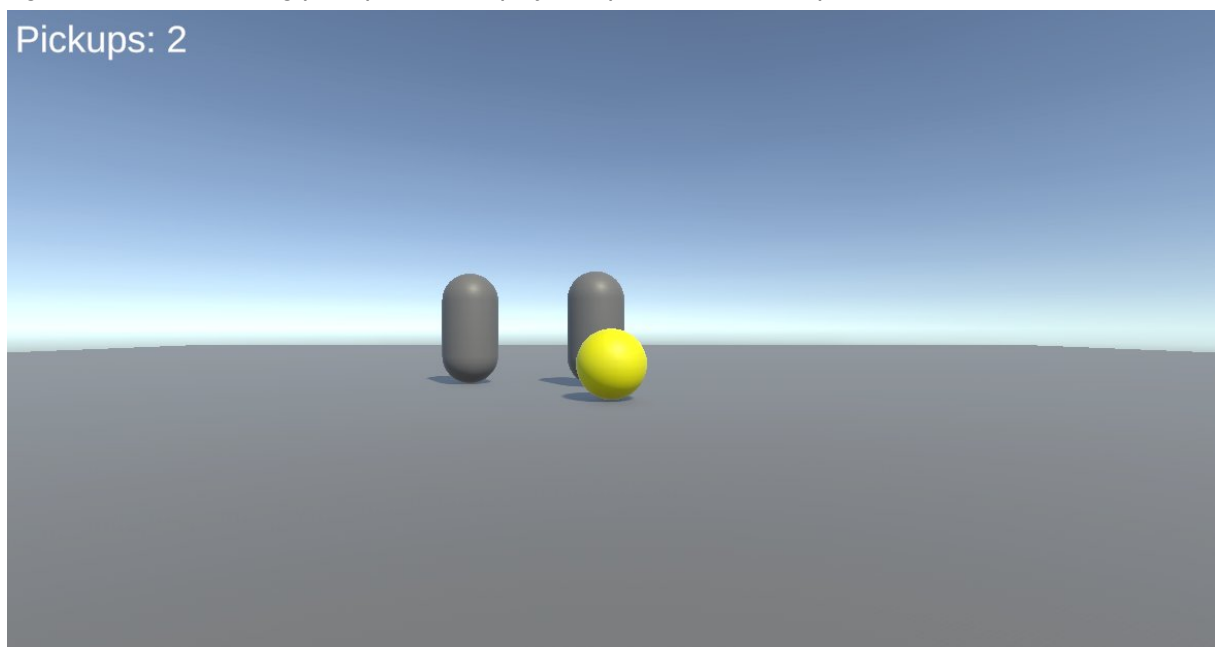
```
25
26     //called on the host when a player collider enters trigger
27     private void OnTriggerEnter(Collider other)
28     {
29         //only the host processes pickups to prevent dupe pickups
30         if (!Object.HasStateAuthority) return;
31
32         if (IsPickedUp) return;
33
34         //if the collider belongs to a player
35         PlayerScore playerScore = other.GetComponent<PlayerScore>();
36         if (playerScore != null)
37         {
38             IsPickedUp = true;
39             playerScore.Score += 1;
40         }
41     }
42 }
```

[Networked] properties were chosen over RPCs because `IsPickedUp` and `Score` are persistent state read every frame in `Render()` to update visuals and UI. A [Networked] property provides the current value to all clients including late joiners automatically. An RPC fires once and would be missed by anyone connecting after the event, leaving them with stale visual state. RPCs would be appropriate for transient one-off effects like a pickup sound or particle burst where only currently connected clients need to react, but there are no transient effects in this scope.

Two clients side by side. Left (editor) shows Pickups: 1, right (build) shows Pickups: 2. Orbs collected and hidden on both screens.



Single client view showing pickup orb, both player capsules, and Pickups: 2 score UI.



Test Matrix

#	Scenario	Expected	Actual	P/F
1	Host starts session	Session created, capsule spawns	Works as expected	Pass
2	Client joins	Second capsule on both screens	Both visible	Pass
3	Host picks up orb	Orb gone on both, host score up	Confirmed	Pass
4	Client picks up orb	Orb gone on both, client score up	Confirmed	Pass
5	Simultaneous pickup	Only one collects	Host first, no double	Pass
6	Already-collected orb	Nothing happens	No effect	Pass
7	All orbs collected	All hidden, scores correct	Consistent	Pass
8	Host WASD	Synced via NetworkTransform	Synced	Pass
9	Client WASD	Synced via NetworkTransform	Synced	Pass
10	Client disconnects	Despawned on host	Removed	Pass

A3 Unreal Replication Terminology

Fusion splits authority into two roles. `StateAuthority` is the entity permitted to write to `[Networked]` properties, which in a Host/Client topology is always the host. `InputAuthority` designates which client provides input for a given `NetworkObject` so each player only controls their own character. In the Spawner script `OnPlayerJoined` calls `Runner.Spawn()` with the joining player as the fourth parameter, which assigns `InputAuthority` to that client.

Unreal Engine 5 maps these concepts to three roles. The Server holds authority over all replicated actors and is the only entity that can authoritatively modify replicated state. This is functionally identical to Fusion's `StateAuthority`. The Owning Client owns a `PlayerController` and its associated Pawn and is the only client permitted to send input to the server for that actor. This maps directly to Fusion's `InputAuthority`. Simulated Proxies are all other clients observing a remotely-controlled actor. They receive replicated state but cannot write to it or provide input. Fusion does not have an explicit Simulated Proxy role. Instead any client without `InputAuthority` on an object simply receives its replicated state through `[Networked]` properties and `NetworkTransform`.

Fusion's `[Networked]` attribute marks a property for automatic synchronisation from `StateAuthority` to all clients. In the `PickupItem` script `[Networked] public NetworkBool IsPickedUp` is automatically replicated whenever the host changes it. Unreal's equivalent is a `UPROPERTY` marked with the `Replicated` specifier, combined with registering the property in `GetLifetimeReplicatedProps()` using the `DOREPLIFETIME` macro. Both achieve the same result but Unreal requires additional boilerplate through the registration macro and the function override, where Fusion handles this through code generation at compile time. Unreal also offers `RepNotify` which triggers a callback function on clients when a replicated property changes. Fusion has a similar mechanism with the `OnChangedRender` attribute, though it was not used in this implementation since the property value is checked directly in `Render()` each frame.

Fusion supports RPCs that can target `StateAuthority`, `InputAuthority`, or all clients. Unreal provides three RPC types that map to these. A Server RPC marked with `UFUNCTION(Server)` is called on a client and executed on the server, equivalent to a Fusion RPC targeting `StateAuthority`. A Client RPC marked with `UFUNCTION(Client)` is called on the server and executed on the owning client, mapping to a Fusion RPC targeting `InputAuthority`. A `NetMulticast` RPC marked with `UFUNCTION(NetMulticast)` is called on the server and executed on all connected clients, equivalent to a Fusion RPC targeting all. In both engines RPCs should be used for transient events like sound triggers or visual effects rather than as a substitute for replicated state, since RPCs are fire-and-forget and will not be received by late joiners.

The most significant conceptual difference is the simulation model. Fusion runs a tick-based deterministic simulation where all networked logic must go in `FixedUpdateNetwork()` at a fixed tick rate. Clients can run prediction ahead of the server. Unreal uses an actor-based replication model where the server replicates entire actor states at a variable rate determined by `NetUpdateFrequency` and clients do not predict by default. In Fusion placing game logic in `Update()` instead of `FixedUpdateNetwork()` breaks determinism and causes inconsistencies between clients. In Unreal game logic runs in `Tick()` on both server and client and replication handles synchronisation separately. Porting networking code between engines would mean rethinking where logic lives rather than just translating API names.

B Version Control Evidence

Commit graph showing feature/ca2-network branch, main branch, milestone tags (baseline, ca1-submit, ca2-baseline, ca2-submit), and AI module branches.

```
* 10ab101 (HEAD -> feature/ca2-network, origin/feature/ca2-network) created orb prefab for pickup
* dfc71f6 added basic pickup with network state and playerscore ui
* ae9e951 (tag: ca2-baseline) two client baseline connect and player spawn work
* 963daa5 Spawner with session setup and player spawn and despawn
* 752520e playercharacter network prefab
* 13c977b CA2 network test scene and ground plane
* c248622 import Photon Fusion 2 SDK and App ID config
* 035725d CA2 network documentation
| * f0fa163 (tag: ca2-submit, origin/CA2, CA2) Rename CriticalDiscussion.md to CA2_CriticalDiscussion.md
| * daacf48 Revise CA2 Evidence Log with session updates
| * 69cdba2 Add Critical Discussion on AI Design and Lessons Learned
| * d88e639 Add technical note on guard AI behavior tree and perception
| * ba70614 Merge pull request #1 from Wolfgang-Romanowski/main
| |
| | * f332baa Add files via upload
| | /
| /
| * 76b6b60 revamped debug overlay and fixed path switch causing AI to alternate too much
| * 7da63bc fixed chase timer, stuck detection, stale blackboard clear
| * 5a6cabf BT leafs and behaviour
| * 970b8ef remove vscode folder
| * d0d20c1 Add BT core classes: Node, Sequence, Selector, decorators, and Blackboard
| * 2570747 Remove .vscode files
| * 5b0a1b8 CA2: ca1 bug fixes
| /
* 3e11c27 (tag: ca1-submit, origin/main, main) add build
* 9798153 add build and remove vscode
* 35dcc79 final submission with docs and build
* 4ee5ce5 remove unnecessary files
* 7557aeb (tag: baseline) ca1 submission
```

For CA1 all commits went directly to main in a single session. For CA2 a feature/ca2-network branch was created from main to isolate all Fusion networking work. All networking scripts, prefabs, scene changes, and documentation were committed on this branch. Once stable the branch was merged back to main and the ca2-submit tag applied.

CA2 commits use descriptive prefixes: net: for networking code, docs: for documentation, fix: for bug fixes. Tags applied: baseline, ca1-submit, ca2-baseline, ca2-submit.

Learning Journal

Evidence Log

Weeks 1-5 (19 Jan - 1 Mar): No commits for the 3D module.

Week 6 (2-8 Mar): CA1 completed and submitted in a single session on March 2nd. Included project setup, PBR imports, Shader Graph dissolve, Unreal comparison, lighting, post-processing, PDF and build. Commits: 7557aeb, 4ee5ce5, 35dcc79, 9798153, 3e11c27 (all 2026-03-02).

Weeks 7-8 (9-22 Mar): No 3D module commits.

Week 9 (23 Mar - 5 Apr): All CA2 networking done in a single session on April 4th. Imported Fusion 2 SDK, configured Photon App ID. Created CA2_NetworkTest scene. Built PlayerCharacter prefab with NetworkObject and NetworkTransform. Wrote Spawner implementing INetworkRunnerCallbacks with Runner.Spawn() in OnPlayerJoined and Runner.Despawn() in OnPlayerLeft. Two-client baseline confirmed.

Built PickupItem with [Networked] NetworkBool IsPickedUp and HasStateAuthority guard in OnTriggerEnter. Built PlayerScore with [Networked] int Score and HasInputAuthority-gated UI. Created PickupOrb prefab with trigger collider. Commits: 035725d, c248622, 13c977b, 752520e, 963daa5, aede951, dfc71f6, 10ab101 (all 2026-04-04).

Blockers: prefab not registered in NetworkProjectConfig, fixed via Rebuild Prefab Table. InputSystem conflict where OnInput used Input.GetKey() but project used new Input System, threw InvalidOperationException, fixed by setting Active Input Handling to Both. OnTriggerEnter not firing due to missing Rigidbody on player, fixed by adding kinematic Rigidbody with gravity disabled.

Critical Discussion

Option B (networked pickup) was chosen because it has a clear binary success condition that is straightforward to verify in a two-client test: either the orb disappears on both screens and the score increments correctly, or it does not. The feature naturally requires both [Networked] properties and StateAuthority guards which are the core concepts being assessed. Option D (networked projectile) was explicitly rejected because reliable hit detection and latency compensation would have introduced significant debugging complexity that could not be confidently resolved within the available time. Even with the simpler pickup scope, multiple blockers consumed development time including the InputSystem conflict, the missing prefab registration in NetworkProjectConfig, and the missing Rigidbody for trigger detection. A larger scope would not have been completed.

[Networked] properties were used for both IsPickedUp (NetworkBool on PickupItem) and Score (int on PlayerScore) instead of RPCs. The choice follows directly from how the code consumes these values. In PickupItem.Render() every client checks IsPickedUp every frame to decide whether to enable or disable the orb's renderer and collider. In PlayerScore.Render() the local client reads Score every frame to update the TextMeshPro UI element. Both values represent persistent state that any client needs access to at any point, including clients that join mid-session. A [Networked] property is the correct tool for this because Fusion automatically replicates the current value to all connected clients including late joiners. If an RPC had been used to broadcast a pickup event, any client connecting after that RPC fired would never receive it and would see the orb still visible despite it being collected on other clients. RPCs would be the appropriate choice for transient one-off events such as triggering a pickup sound effect or particle burst where only clients present at that exact moment need to react. There were no transient effects in this feature scope so RPCs were not needed.

The most important authority model decision was in PickupItem.OnTriggerEnter(). The first implementation had no authority check. During two-client testing both the host and the joining client processed the collision

at the same time. This meant `IsPickedUp` was set to true on both clients independently and `playerScore.Score += 1` executed on both sides, doubling the score for a single pickup. The fix was adding `if (!Object.HasStateAuthority) return;` as the first line of `OnTriggerEnter`, ensuring that only the host (which holds `StateAuthority` over scene-placed `NetworkObjects`) processes the pickup collision and writes to the `[Networked]` properties. The joining client's collision is effectively ignored and it receives the updated `IsPickedUp` and `Score` values through Fusion's automatic replication. This directly matches the common pitfall described in the rubric: applying state changes on all clients simultaneously is the most common source of persistent desyncs.

If starting over at the beginning of Week 8, the main change would be testing with two clients from the very first hour rather than building all scripts before testing. The three biggest blockers during development were the `InputSystem` exception, the missing prefab registration, and the missing `Rigidbody` for trigger collisions. Each was a fast fix once identified but all three only surfaced during two-client runtime testing. Because testing was deferred until after all scripts were written, these issues were discovered and resolved sequentially rather than in parallel. Starting with a minimal two-client spawn test immediately would have caught the `InputSystem` and prefab registration issues within the first session, leaving more time for the actual feature implementation.

Lessons Learned

1. Prefab registration is a silent failure. The `PlayerCharacter` prefab had `NetworkObject` but was not in the `NetworkProjectConfig` prefab table. Fusion threw no error. The object did not appear on the second client. `Rebuild Prefab Table` fixed it instantly.
2. Unity requires a `Rigidbody` on at least one object in a trigger collision pair. The `PickupOrb` had a trigger collider and the player had a `CapsuleCollider` but `OnTriggerEnter` never fired. Adding a kinematic `Rigidbody` with gravity disabled resolved it.
3. Every write to a `[Networked]` property needs a `HasStateAuthority` guard. Without the check both clients processed the collision and doubled the score. Adding `if (!Object.HasStateAuthority) return;` fixed the desync.
4. Compressing an entire assessment into one session leaves no time for iteration or edge case testing. Both CA1 and CA2 were done in single days and the commit histories reflect that.
5. The old and new `Input Systems` conflict at runtime without warning. The project used the new `Input System` but `OnInput` used `Input.GetKey()`. This threw an `InvalidOperationException` at runtime with no compile error. Setting `Active Input Handling` to `Both` resolved it.

Attribution

Photon Fusion 2 SDK (v2.0.12) — Photon Engine, <https://www.photonengine.com/fusion> — Free tier

TextMeshPro — Unity Technologies (included via Unity Package Manager)